Objektorientierte Programmierung mit Ruby on Rails

2-Wochen-Aufgabe für Azubi Anwendungsentwicklung

Projekt: Netzdoku

Du baust in den nächsten zwei Wochen Schritt für Schritt eine kleine Verwaltungsanwendung für Netzwerk-Equipment: Kunden, Verträge, Geräte (Router, Switches, Access Points). Jeder Tag baut auf dem Code des Vortages auf – arbeite also sauber und nachvollziehbar, du brauchst deinen eigenen Code später wieder.

Beantworte alle Theoriefragen **schriftlich, in eigenen Worten und immer mit einem konkreten Beispiel aus deinem eigenen Code** – nicht mit allgemeinen Lehrbuchdefinitionen. Halte deine Antworten in einer Datei `theorie.md` im Projekt fest.

Woche 1 – Grundlagen

Tag 1: OOP-Grundlagen in purem Ruby (noch ohne Rails)

Aufgabe: Baue ohne Rails, nur in normalen Ruby-Dateien, folgende Klassenhierarchie:

- Basisklasse `NetworkDevice` mit Attributen `name`, `ip_address`, `status`
- Unterklassen `Router`, `Switch`, `AccessPoint`, die jeweils eigene, spezifische Attribute/Methoden ergänzen (z. B. `Router#routing_table`, `Switch#port_count`)
- Jede Klasse implementiert eine Methode `describe`, die je nach Gerätetyp unterschiedlich formatiert ausgibt (**Polymorphismus**)
- Kapselung: Der Status darf **nicht** direkt über `attr_accessor` verändert werden, sondern nur über eine Methode `update_status!`, die ungültige Werte abweist

Theoriefragen: 1. Was ist der Unterschied zwischen einer Klasse und einem Objekt? Zeig es an deinem `Router`-Beispiel. 2. Wo genau in deinem Code siehst du Kapselung? Was würde passieren, wenn du stattdessen `attr_accessor :status` genutzt hättest? 3. Erkläre Vererbung vs. Komposition an einem eigenen Beispiel – wäre `AccessPoint erbt von NetworkDevice` in jedem Fall die beste Lösung, oder hätte Komposition (`has_a` statt `is_a`) an einer Stelle mehr Sinn ergeben?

Tag 2: Module & Mixins

Aufgabe: Extrahiere gemeinsames Verhalten (z. B. eine Ping-Simulation oder Logging) in ein Modul `Pingable`, das per `include` in mehrere deiner Geräteklassen eingebunden wird.

Theoriefragen: 1. Warum hat Ruby keine Mehrfachvererbung, aber Module? Was ist der Unterschied zu Interfaces in Java/C#? 2. Wann würdest du ein Modul statt einer Vererbungshierarchie wählen?

Zusatzaufgabe (ohne Rechner, auf Papier): Zeichne die Methodenauflösungsreihenfolge (Method Resolution Order) für eine deiner Klassen, die zwei Module einbindet. Lege das Blatt zu deiner Abgabe.

Tag 3: Einstieg Rails – ActiveRecord ist auch nur Ruby

Aufgabe: `rails new netzdoku`. Modelliere `Customer`, `Contract`, `Device` als ActiveRecord-Modelle mit sinnvollen Assoziationen (`has_many`, `belongs_to`).

Theoriefragen: 1. Was übernimmt ActiveRecord von der objektorientierten Modellierung, und was ist reine "Datenbank-Magie"? 2. Was ist der Unterschied zwischen einem PORO (Plain Old Ruby Object) und einem ActiveRecord-Modell? Wann würdest du bewusst kein ActiveRecord-Modell verwenden?

Tag 4: Vererbung in Rails – Single Table Inheritance (STI)

Aufgabe: Bilde deine `Router`/`Switch`/`AccessPoint`-Hierarchie aus Tag 1 jetzt als STI in `Device` ab. Baue bewusst einen Fall ein, bei dem STI an seine Grenzen stößt (z. B. ein Attribut, das nur für `Router` Sinn ergibt).

Theoriefragen: 1. Was sind die Nachteile von STI? Wann wäre eine polymorphe Assoziation (`polymorphic: true`) die bessere Wahl? 2. Verstößt STI in deinem Fall gegen das Liskov-Substitutionsprinzip? Begründe konkret an deinem Beispiel.

Tag 5: Debugging-Tag

Aufgabe: Nimm dir deinen eigenen Code der letzten vier Tage und suche dir bewusst *drei Stellen*, an denen ein typischer OOP-Fehler passieren könnte (z. B. fehlendes `super`, falsche Assoziationsrichtung, eine Validierung, die durch eine überschriebene Methode nie greift). Baue diese Fehler absichtlich ein, notiere sie **nicht**, warte einen Tag und finde sie am nächsten Morgen selbst wieder.

Dokumentiere in `theorie.md`: - Welcher Fehler wurde eingebaut? - Wie hast du ihn gefunden? - Warum genau hat er diesen Effekt gehabt?

Woche 2 – Vertiefung: Design-Prinzipien & echte Rails-Praxis

Tag 6–7: Service Objects & Single Responsibility

Aufgabe: Schreibe zunächst bewusst eine "Fat Controller"-Action, die Vertragsanlage, Gerätezuweisung und Benachrichtigung in einer einzigen Methode erledigt. Refaktoriere sie anschließend in einzelne Service-Objekte (PORO-Klassen, keine ActiveRecord-Modelle), z. B. `ContractCreator`, `DeviceAssigner`, `NotificationSender`.

Theoriefragen: 1. Welches SOLID-Prinzip wird durch den ursprünglichen "Fat Controller" verletzt? Zeig konkret, welche Zeile welches Prinzip bricht. 2. Was ist der Vorteil von Service Objects gegenüber "Fat Models" oder "Fat Controllers"? Wo liegen die Grenzen (Overengineering)?

Tag 8–9: Polymorphismus & Interfaces im Rails-Kontext

Aufgabe: Baue ein `NotificationChannel`-Konzept: `EmailNotification`, `SlackNotification`, `WebhookNotification` – alle müssen eine gemeinsame Methode `send_message` implementieren, ohne dass der aufrufende Code wissen muss, um welchen Typ es sich handelt (Duck Typing).

Erweitere dein Konzept anschließend spontan um einen vierten Kanal (`SmsNotification`), ohne bestehenden Code außer der neuen Klasse selbst anzufassen.

Theoriefragen: 1. Ruby hat keine expliziten Interfaces wie Java. Wie stellst du trotzdem sicher, dass alle `Notification`-Klassen die erwartete Methode implementieren? Diskutiere Duck Typing vs. das `NotImplementedError`-Pattern in der Basisklasse. 2. Was ist Duck Typing, und warum passt der Name? 3. Warum ließ sich `SmsNotification` ergänzen, ohne bestehenden Code zu ändern? Welches SOLID-Prinzip zeigt sich hier?

Tag 10: Tests schreiben & Abschlussreflexion

Aufgabe: Schreibe RSpec-Unit-Tests für mindestens drei deiner selbst geschriebenen Klassen (nicht die ActiveRecord-Modelle, sondern deine PORO-/Service-Klassen). Die Tests müssen mindestens einen Edge Case abdecken.

Abschlussfragen (schriftlich, frei formuliert): 1. Welche der 10 Aufgaben hat dir das Konzept “Objektorientierung” am klarsten gemacht, und warum? 2. Wenn du das Projekt nochmal von vorne beginnen würdest – was würdest du objektorientiert anders lösen? 3. Nenne drei konkrete Design-Entscheidungen aus deinem Projekt und begründe jeweils in 2–3 Sätzen, warum du dich so entschieden hast.

Zusammenfassung der Theoriefragen (Nachschlageliste)

- Klasse vs. Objekt
- Kapselung – Sinn und Umsetzung in Ruby
- Vererbung vs. Komposition (“is-a” vs. “has-a”)
- Module/Mixins vs. Mehrfachvererbung vs. Interfaces
- Polymorphismus und Duck Typing
- SOLID-Prinzipien (mind. SRP, LSP, ISP an konkreten Beispielen)
- PORO vs. ActiveRecord-Modell – wann was?
- STI vs. polymorphe Assoziation
- Service Objects – Nutzen und Grenzen
- Method Resolution Order in Ruby

Abgabe

Reiche am Ende der zwei Wochen ab:

- Den vollständigen Projekt-Code (Git-Repository oder Ordner)
- Die Datei `theorie.md` mit allen Antworten
- Das Papier-Blatt aus der MRO-Zusatzaufgabe (Tag 2)

Viel Erfolg!